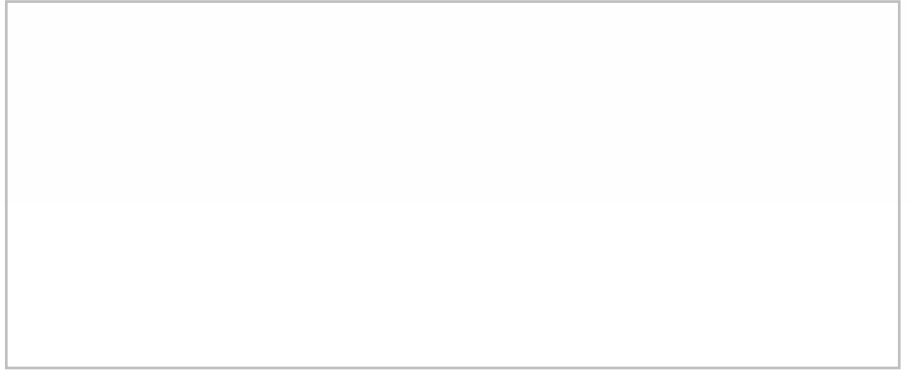




Ne rien inscrire dans ce cadre

Prénom

Nom

Promotion

Groupe

Promotion : Ing1 – LSI - Bordeaux

Module : Programmation avancée en Java

ALS162B-DE/2425S6

1h50 Horaire : 15H20 – 17H10

Examen sur table

Sujet rédigé par : Monsieur Jean-Aymeric DIET

Supports autorisés :

Calculatrice autorisée : **non**

Documents autorisés : non

Traducteur électronique : non

Dictionnaire : non

Consigne :

Merci de restituer : **le sujet avec votre copie quadrillée**

Rappel :

- Tous les appareils électroniques (téléphones portables, ordinateurs, tablettes, montres connectées, accès à internet ...) doivent être éteints et rangés.
- Il est interdit de communiquer.
- Toute fraude ou tentative de fraude fera l'objet d'un rapport de la part du surveillant et sera sanctionnée par la note zéro, assortie d'une convocation devant le conseil de discipline. Aucune contestation ne sera possible. Tous les documents et supports utilisés frauduleusement devront être remis au surveillant.
- Aucune sortie de la salle d'examen ne sera autorisée avant la moitié de la durée de l'épreuve.

Ne rien inscrire dans ce cadre

Partie 1

Après lecture de l'interface suivante :

Transformer.java

```
@FunctionalInterface
public interface Transformer<Input, Output>{
    Output apply(Input input);
}
```

Créez une classe StringLengthTransformer implémentant Transformer<String, Integer> et qui retourne la longueur d'une chaîne de caractères

```
Public class StringLengthTransformer implements Trasnformer<String, Integer>{

    @override

    Integer apply(String input){

        Return input.length()

    }

}
```

Créez un objet stringLengthTransformer réalisant la même chose mais en utilisant une expression lambda

```
StringLengthTransformer str = new StringLengthTransformer((String s) -> s.length());
```

Créez une classe générique GenericTransformer qui prend un Transformer<T, R> dans son constructeur et utilise apply() pour transformer un élément.

```
public class GenericTransformer<T,R> {
    private Transformer<T, R> transformer;
    public GenericTransformer(Transformer<T, R> transfromer) {
        this.transformer = transfromer;
    }
}
```

Créez une énumération Transformations possédant :

- TO_UPPERCASE : transformation d'une chaîne en majuscule
- TO_LOWERCASE : transformation d'une chaîne en minuscule

Ajoutez une méthode apply() permettant de masquer l'appel à apply() de l'interface.

```
public enum Transformations {
    TO_UPPERCASE((String s) -> s.toUpperCase()),
    TO_LOWERCASE((String s) -> s.toLowerCase());

    private Transformer<String, String> genericTransformer;
```

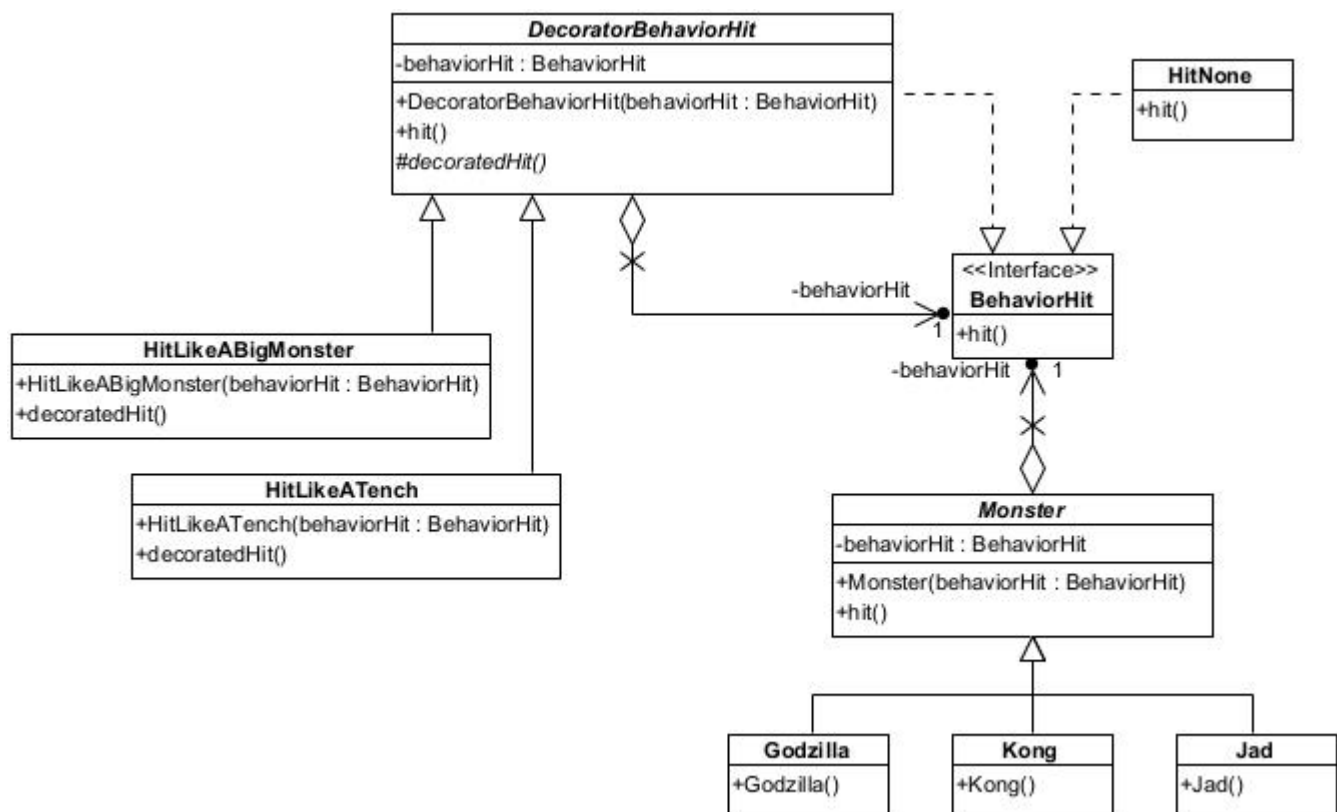
```

private Transformations(Transformer<String, String> genericTransformer){
    this.genericTransformer = genericTransformer;
}

public String apply(String input) {
    return genericTransformer.apply(input);
}
}

```

Partie 2



Après lecture du diagramme de classes, codez la classe **DecoratorBehaviorHit**.

DecoratorBehaviorHit.java

```

package partie2;

public abstract class DecoratorBehaviorHit {
    private BehaviorHit behaviorHit;

    public DecoratorBehaviorHit(BehaviorHit behaviorHit) {
        this.behaviorHit = behaviorHit;
    }

    public void hit(){

```

```
        this.behaviorHit.hit();
        this.decoratedHit();
    }

    protected abstract void decoratedHit();
}
```

Après lecture du diagramme de classes, codez la classe Monster

Monster.java

```
package partie2;

public abstract class Monster {
    private BehaviorHit behaviorHit;

    public Monster(BehaviorHit behaviorHit) {
        this.behaviorHit = behaviorHit;
    }

    public void hit(){
        this.behaviorHit.hit();
    }
}
```

Après lecture du diagramme de classes, codez la classe Jad, en sachant qu'un objet issu de la classe Jad par défaut tape comme une tanche.

Jad.java

```
package partie2;

public class Jad extends Monster{
    public Jad() {
        super(new HitLikeATench(new HitNone()));
    }
}
```

Après lecture du diagramme de classes, expliquez ce qu'il faudrait faire pour permettre à un objet déjà instancié de la classe Jad, de taper comme une tanche, mais aussi comme un gros monstre.

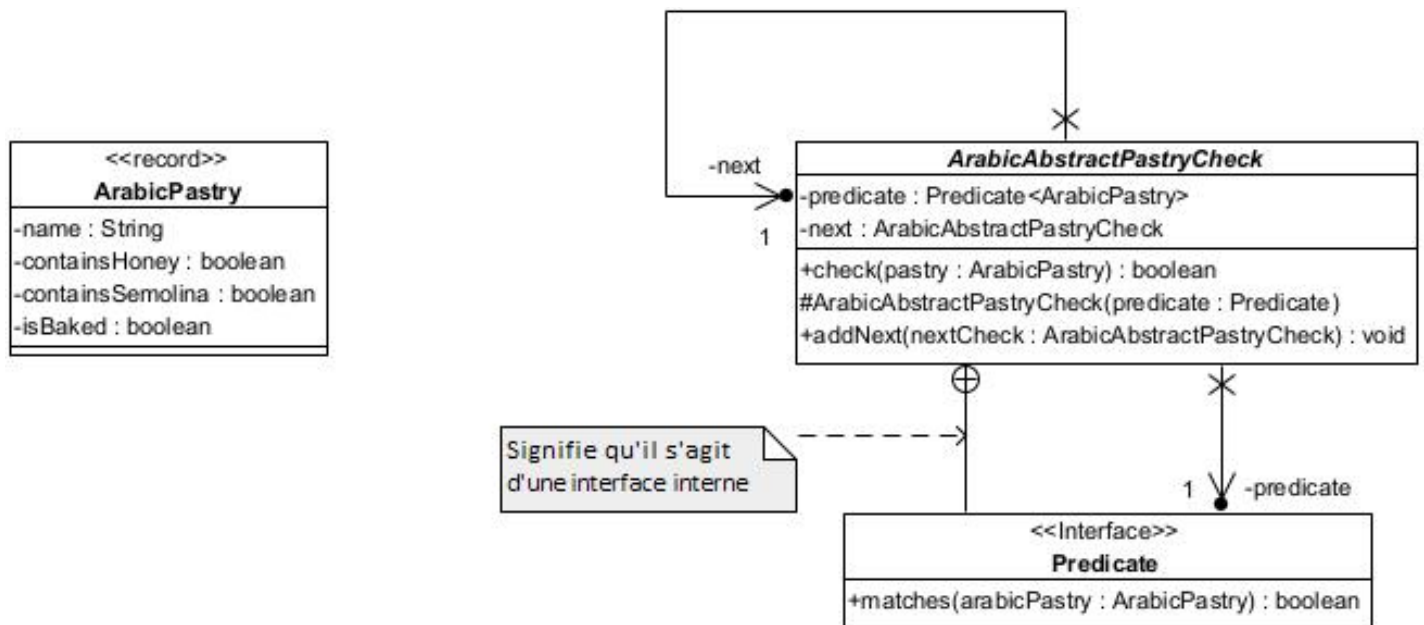
Rajouter un setBehaviorHit dans le Monster puis faire
monster1.setBehaviorHit(new HitLikeATench(new HitLikeABigMonster(new HitNone())));

Codez ce qu'il faudrait taper dans le main, pour donner ce nouveau comportement à un objet de classe Jad.

```
public class Main {
    public static void main(String[] args) {
        Monster monster1 = new Jad();
        monster1.hit();
        monster1.setBehaviorHit(new HitLikeATench(new HitLikeABigMonster(new HitNone())));
        monster1.hit();
    }
}
```

```
}  
}
```

Partie 3



Après analyse du diagramme de classe, codez le record **ArabicPastry**

```
public record ArabicPastry(String name, Boolean containsHoney, Boolean containsSemolina, boolean isBaked) {}
```

Après analyse du diagramme de classe, codez l'interface **Predicate**. Un **Predicate** va vérifier (via la méthode **matches**, qu'un objet de type **ArabicPastry** vérifie une condition.

```
public interface Predicate {  
    public boolean matches(ArabicPastry arabicPastry);  
}
```

Coder la classe **ArabicAbstractPastryCheck** (comme son nom l'indique, elle implémente l'interface **ArabicPastryCheck** et est abstraite. Il suffit qu'un check de la chaîne renvoie faux pour stopper le parcours de la chaîne. La méthode **check** doit obligatoirement être finale.

ArabicAbstractPastryCheck.java

```
public abstract class ArabicAbstractPastryCheck {  
    private Predicate predicate;  
    private ArabicAbstractPastryCheck next;  
  
    protected ArabicAbstractPastryCheck(Predicate predicate) {  
        this.predicate = predicate;  
    }  
  
    public boolean check(ArabicPastry pastry) {  
        return this.predicate.matches(pastry);  
    }  
}
```

```
public void addNext(ArabicAbstractPastryCheck nextCheck){  
    this.next = nextCheck;  
}  
}
```

Coder la classe HoneyCheck qui hérite de ArabicAbstractPastryCheck et vérifie que la pâtisserie arabe contient du miel. Utilisez une expression lambda.

HoneyCheck.java

```
public class HoneyCheck extends ArabicAbstractPastryCheck{  
  
    protected HoneyCheck() {  
        super((arabicPastry) -> arabicPastry.containsHoney());  
    }  
}
```

Considérez les classes SemolinaCheck et BakedCheck implémentée, codez la classe MakroutCheck qui utilise les 3 checks. MakroutCheck doit aussi hériter de ArabicAbstractPastryCheck.

MakroutCheck.java

```
public class MakroutCheck extends ArabicAbstractPastryCheck{  
    protected MakroutCheck() {  
        super((a) -> new HoneyCheck().check(a) && new SemolinaCheck().check(a) && new BakedCheck().check(a));  
    }  
}
```